

HTTP, HTTPS, Stateful vs Stateless, Sessions, and JWT Authentication

Introduction

When building web applications, it is important to understand:

- HTTP and HTTPS
- Stateless vs Stateful systems
- Cookies and Sessions
- JWT Authentication
- Registration and Login flows

These concepts form the foundation of modern web authentication and authorization systems.

1. HTTP and HTTPS

HTTP (Hypertext Transfer Protocol)

HTTP is the protocol used for communication between a browser (client) and a web server.

HTTP is **stateless**, meaning the server does not automatically remember previous requests.

Example

```
Browser — Request —> Server  
Browser <— Response — Server
```

HTTPS (Hypertext Transfer Protocol Secure)

HTTPS is HTTP running over SSL/TLS encryption.

HTTPS provides:

- Confidentiality
- Integrity
- Authentication

Example

```
Browser — Encrypted Request —> Server  
Browser <— Encrypted Response — Server
```

Important

```
HTTP  = Stateless  
HTTPS = Stateless + Encryption
```

2. What Does Stateless Mean?

A stateless system does not remember previous interactions.

Every request is treated as a completely new request.

Example

Request 1:

```
GET /profile
```

Request 2:

```
GET /orders
```

The server does not automatically know the same user made both requests.

Real-Life Analogy

A cashier with no memory:

```
Customer: I want a coffee.  
Customer: I want a sandwich.  
Cashier: Who are you again?
```

3. What Does Stateful Mean?

A stateful system remembers previous interactions.

Example: Phone Call

```
You: Hi  
Friend: Hi
```

```
You: How was your exam?  
Friend: It went well.
```

The friend remembers previous context.

Real-Life Analogy

```
Barber: Welcome back!  
Same haircut as last time?
```

4. Stateful vs Stateless

Feature	Stateful	Stateless
Remembers previous requests	Yes	No
Stores client state	Server	Client/Request
Server memory needed	Yes	Usually No
Scalability	Lower	Higher
Logout	Easy	Harder
Examples	Sessions	HTTP, JWT

Memory Trick

```
Stateful   = I remember you.  
Stateless  = Tell me who you are every time.
```

5. How Websites Remember Users

Since HTTP and HTTPS are stateless, applications use additional mechanisms:

- Cookies
- Sessions
- JWT Tokens
- URL Rewriting

Cookies

A cookie is small data stored in the browser.

Example:

```
SESSIONID=S123
```

The browser automatically sends it with future requests.

Sessions

A session stores user information on the server.

Example:

```
SESSIONID S123 → Alice
```

The browser only stores the session ID.

JWT Tokens

JWT stands for JSON Web Token.

Example:

```
{  
  "sub": "123",  
  "name": "Alice",  
  "role": "admin"  
}
```

The token contains user information and is sent with each request.

6. What Is a Session?

A session is a way for a server to remember who you are across multiple requests.

Since HTTP is stateless, the server forgets everything after sending a response. A session solves this problem.

Simple Definition

A session is a temporary connection between a user and a server where the server stores information about that user.

Example: Logging Into a Website

Step 1: Login

```
Username: alice  
Password: 12345
```

Step 2: Server Verifies You

```
Users Table  
-----  
alice   hashed_password
```

If credentials are valid:

```
Session ID = S123
```

Step 3: Server Stores the Session

```
Session Store  
-----  
S123 → User: Alice
```

Meaning:

```
If I see S123 again, I know this is Alice.
```

Step 4: Session ID Sent to Browser

```
Set-Cookie: SESSIONID=S123
```

Step 5: Future Requests

Browser sends:

Cookie: SESSIONID=S123

Server checks:

S123 → Alice

Response:

Welcome back, Alice!

Visual Flow

Login

```
Browser -----> Server
      username/password
```

```
Browser <----- Server
      SESSIONID=S123
```

Session Store

S123 → Alice

Next Request

```
Browser -----> Server
Cookie: SESSIONID=S123
```

```
Server:
S123 → Alice
```

```
Response:
Welcome back, Alice!
```

What Does a Session Store?

Example:

```
S123 → {  
  userId: 42,  
  username: "Alice",  
  isLoggedIn: true,  
  cartItems: 3  
}
```

A session may store:

- User ID
- Username
- Login status
- Shopping cart items
- User preferences

Real-Life Analogy

Restaurant cloakroom:

Real World	Session System
Coat	User Data
Token #25	Session ID
Storage Room	Session Store
Showing Token	Sending Session ID

Key Idea

Session = Server remembers you using a Session ID.

7. Session-Based Authentication (Stateful)

How It Works

1. User logs in.
2. Server creates a session.
3. Server stores session data.
4. Browser receives a session ID cookie.
5. Browser sends the session ID with future requests.
6. Server checks the session store.

Flow

```
Browser — Login → Server
Browser ← Session Cookie — Server

Future Requests:

Browser — Session ID → Server
Server — Checks Session Store → Authenticated
```

Advantages

- Easy logout
- Easy session invalidation
- Sensitive data stays on the server

Disadvantages

- Requires session storage
- Harder to scale without shared session storage

8. JWT Authentication (Stateless)

JWT = JSON Web Token

Structure:

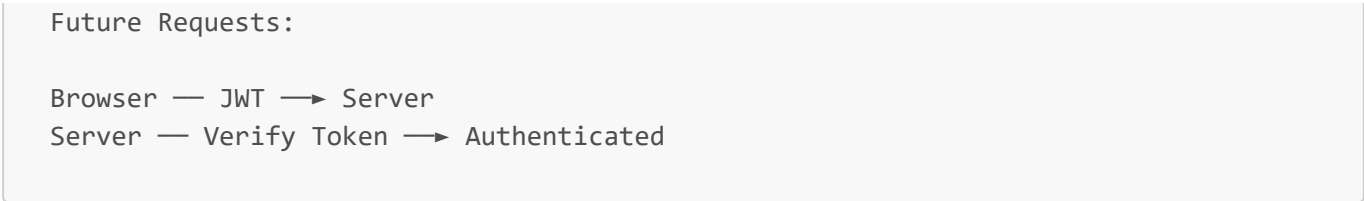
```
Header.Payload.Signature
```

Example Payload

```
{
  "sub": "123",
  "name": "Alice",
  "role": "admin",
  "exp": 1780000000
}
```

Flow

```
Browser — Login → Server
Browser ← JWT Token — Server
```

Advantages

- No session storage required
- Highly scalable
- Great for APIs and microservices

Disadvantages

- Logout is harder
- Revocation is difficult
- Larger request size

9. Session vs JWT Comparison

Feature	Session Authentication	JWT Authentication
User data stored	Server	Inside token
Client stores	Session ID	JWT
Server state	Stateful	Stateless
Verification	Session lookup	Signature verification
Scalability	Lower	Higher
Logout	Easy	Harder
Revocation	Immediate	Difficult
Token size	Small	Larger
Common Use Cases	Traditional Websites	APIs, SPAs, Mobile Apps, Microservices

10. Which Authentication Method Should You Use?

Use Sessions For

- Traditional web applications
- Banking applications
- Admin dashboards
- Applications requiring strong session control

Use JWT For

- REST APIs
- React applications
- Mobile apps
- Microservices

Quick Memory Trick

Session = Server remembers you.
JWT = You carry proof of who you are.

11. Registration (Sign Up) Flow

Registration creates a user account.

Step 1: User Provides Data

Username: john
Email: john@example.com
Password: mypassword

Step 2: Validation

Server checks:

- Username uniqueness
- Email uniqueness
- Password rules

Step 3: Password Hashing

```
mypassword
  ↓
bcrypt
  ↓
$2b$12$X7k...
```

Common algorithms:

- bcrypt
- Argon2
- PBKDF2

Step 4: Store User

User ID	Username	Email	Password Hash
42	john	john@example.com	\$2b\$12\$X7k...

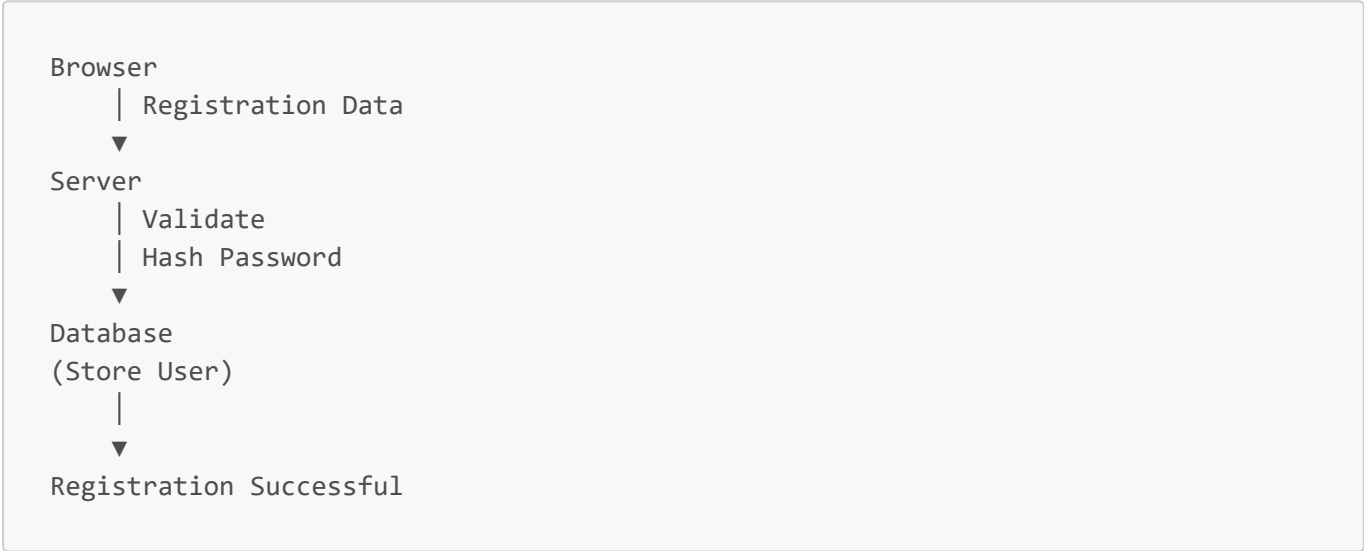
Step 5: Success Response

Either:

Registration Successful

or automatically log the user in.

12. Registration Without Auto Login



13. Registration With Auto Login



14. Login Flow

Session Login

1. User enters credentials
2. Server verifies credentials
3. Session created
4. Session ID sent to browser
5. Browser sends Session ID on every request
6. Server validates Session ID

JWT Login

1. User enters credentials
2. Server verifies credentials
3. JWT created
4. JWT sent to browser
5. Browser sends JWT on every request
6. Server verifies JWT signature

Key Takeaways

- HTTP is stateless.
- HTTPS is stateless and encrypted.
- Stateful systems remember previous interactions.
- Stateless systems do not remember previous interactions.
- Sessions are stateful.
- JWT authentication is stateless.
- Registration creates user accounts.
- Login authenticates users.
- Sessions work best for traditional websites.
- JWT works best for APIs, SPAs, mobile apps, and microservices.

Conclusion

Understanding HTTP, HTTPS, Stateful systems, Stateless systems, Sessions, and JWT authentication is essential for modern web development.

Remember:

Stateful = I remember you.
Stateless = Tell me who you are every time.

Session = Server remembers you.
JWT = You carry proof of who you are.